

实验十一 Keras 模块的使用

（一）目的和要求

- （1）理解 Keras 框架的基本架构与工作原理。
- （2）掌握 Keras 模型构建方法。
- （3）了解数据预处理与增强的方法。
- （4）掌握模型训练与调优的方法。
- （5）掌握模型保存与加载的方法

（二）实验知识准备

（1）什么是 Keras

Keras 是一个高级神经网络 API，它允许快速和简单地设计和训练深度学习模型。它旨在减少深度学习研究和应用中的繁琐工作，通过提供易于使用且模块化的接口，使得研究人员和开发人员能够更专注于模型的构建和实验。其主要功能有：

1. **模型构建：**Keras 提供了多种模型构建方式，包括 Sequential 模型和 FunctionalAPI。Sequential 模型允许用户按顺序添加层来构建模型，而 FunctionalAPI 则提供了更灵活的模型构建方式，支持复杂的神经网络架构。

2. **层与激活函数：**Keras 包含了许多常用的神经网络构建块，如全连接层（Dense）、卷积层（Conv2D）、池化层（Pooling）等。此外，它还提供了多种激活函数，如 ReLU、Sigmoid、Tanh 等，可以根据需要选择合适的激活函数。

3. **损失函数与优化器：**Keras 支持多种损失函数和优化器，用户可以根据任务需求选择合适的损失函数和优化器来训练模型。

4. **数据预处理与增强：**Keras 提供了丰富的数据预处理和增强功能，如图像数据的旋转、缩放、裁剪等。

5. **模型评估与调优：**Keras 允许用户轻松地对模型进行评估和调优。用户可以使用测试数据集来评估模型的性能，并通过调整超参数来优化模型。此外，Keras 还支持交叉验证等高级评估技术。

（2）Keras 的应用场景

Keras 广泛应用于各种深度学习领域，包括计算机视觉、自然语言处理、时

间序列分析等。在计算机视觉领域，Keras 被用于图像分类、物体检测、图像生成等任务；在自然语言处理领域，Keras 被用于文本分类、情感分析、机器翻译等任务；在时间序列分析领域，Keras 被用于预测股票价格、分析传感器数据等任务。

（三）实验内容及步骤

（1）利用 Dataset 对象处理数据。当数据集很大时，将数据转换为 tf.data.Dataset 对象，可以方便进行数据处理任务。示例代码如图 1 所示。

```
1 import tensorflow as tf
2 import numpy as np
3
4 dataset = tf.data.Dataset.from_tensor_slices(
5     np.array([1.0, 2.0, 3.0, 4.0, 5.0]))
6
7 for element in dataset:
8     print(element)
```

图 1 Dataset 处理数据

图 1 所示代码中，tf.data.Dataset.from_tensor_slices()是 TensorFlow 提供的一个函数，用于从给定的张量（Tensor）中创建一个数据集（Dataset）。这个函数接受一个或多个张量作为输入，并将它们“切片”成一个数据集，其中每个元素都是输入张量的一个切片。np.array([1.0,2.0,3.0,4.0,5.0])创建了一个包含 5 个浮点数的 NumPy 数组。tf.data.Dataset.from_tensor_slices()将这个 NumPy 数组转换成一个 TensorFlow 数据集，其中每个元素都是数组中的一个浮点数。

（2）读取数据。示例代码如图 2 所示。

```
1 from tensorflow.keras import datasets # 导入经典数据集加载模块
2 import matplotlib.pyplot as plt
3 # 导入数据
4 (X_train, y_train), (X_test, y_test) = datasets.mnist.load_data()
5 # summarize loaded dataset
6 print('训练集 : X={0}, y={1}'.format(X_train.shape, y_train.shape))
7 print('测试集 : X={0}, y={1}'.format(X_test.shape, y_test.shape))
8 # 绘制前4个图片
9 for i in range(4):
10     # 定义子图
11     plt.subplot(2, 2, i+1)
12     # 绘制像素数据
13     plt.title('label = {}'.format(y_train[i]))
14     plt.imshow(X_train[i], cmap=plt.get_cmap('gray_r'))
15     plt.axis('off')
16 # 显示图片
17 plt.show()
```

图 2 读取数据

图 2 所示代码中，这段代码首先从 TensorFlow 的 Keras 库中导入 MNIST 手写数字数据集，并将其分为训练集和测试集。接着，代码打印出训练集和测试集的形状，以使用户了解数据的维度和样本数量。随后，通过循环绘制前四张训练图像，每张图像上方标注了对应的标签。使用 Matplotlib 的 `plt.subplot` 功能将图像安排在 2 行 2 列的布局中，以便于比较。图像以灰度反转显示，增强了数字的可视化效果。最后，调用 `plt.show()` 显示所有绘制的图像，提供了直观的可视化结果，帮助用户理解数据集的特征和样本内容。其中，`.npz` 是 NumPy 库中用于存储数组的压缩文件格式。它可以将多个 NumPy 数组以压缩形式保存到一个文件中，便于高效的存储和加载。

`datasets.mnist.load_data()` 方法会自动下载 MNIST 数据集（如果本地没有的话），并将其加载到内存中。数据集分为训练集 (`X_train, y_train`) 和测试集 (`X_test, y_test`)。 `X_train` 和 `X_test` 是包含图像数据的 NumPy 数组，形状为 (样本数量, 高度, 宽度)，对于 MNIST 数据集，图像大小为 28x28 像素。 `y_train` 和 `y_test` 是包含标签的 NumPy 数组，形状为 (样本数量,)，每个标签对应于图像中的数字。

(3) 使用 `tf.keras` 搭建模型。示例代码如图 3。

```
1 import numpy as np
2 import tensorflow as tf
3 from tensorflow.keras import datasets, Sequential
4 from tensorflow.keras import datasets, layers, Sequential
5
6 # 导入数据
7 (X_train, y_train), (X_test, y_test) = datasets.mnist.load_data()
8
9 X_train = tf.convert_to_tensor(X_train, dtype=tf.float32) / 255.
10 dataset = tf.data.Dataset.from_tensor_slices((X_train, y_train))
11 dataset = dataset.batch(32).repeat(10)
12
13 model = Sequential() #搭建空顺序模型
14 model.add(layers.Flatten(input_shape=in_shape))
15 model.add(layers.Dense(n_classes, activation='softmax'))
```

图 3 keras 搭建模型

图 4 所示代码这段代码首先导入了必要的库，包括 NumPy 和 TensorFlow，以及 Keras 中的数据集和模型构建模块。接着，它使用 `datasets.mnist.load_data()` 方法加载 MNIST 手写数字数据集，将数据分为训练集 (`X_train, y_train`) 和测试集

(X_test,y_test), 其中 X_train 和 X_test 包含图像数据, y_train 和 y_test 包含对应的标签。

随后, 训练集的图像数据通过 `tf.convert_to_tensor` 方法被转换为 TensorFlow 张量, 并将数据类型设置为 `float32`, 同时对像素值进行归一化处理, 使其范围在 0 到 1 之间, 这有助于提高模型的训练效果。

接下来, 使用 `tf.data.Dataset.from_tensor_slices` 将训练数据和标签组合起来, 便于后续的批量处理, 并将数据集分批为大小为 32 的批次, 同时通过 `.repeat(10)` 方法使数据集在训练过程中重复 10 次, 以确保模型能够多次遍历整个训练集。然后, 使用 `Sequential` 类构建了一个顺序模型。首先添加了一个 `Flatten` 层, 该层将每张输入图像展平为一维数组, 以便为后续的全连接层提供输入, 输入形状应由 `in_shape` 变量指定, 该变量通常是一个元组, 表示输入的维度, 例如(28,28)。最后, 代码添加了一个全连接层, 设置输出神经元数量为 `n_classes`, 通常在 MNIST 的情况下为 10, 因为需要分类的数字是 0 到 9, 并且使用 `softmax` 激活函数将输出转化为概率分布, 确保所有输出的和为 1。

(4) 构建梯度递减的手写数字识别。示例代码如图 4。

```

1 import numpy as np
2 import tensorflow as tf
3 from tensorflow.keras import datasets, layers, optimizers, Sequential, metrics
4 # 导入数据
5 (X_train, y_train), (X_test, y_test) = datasets.mnist.load_data()
6
7 X_train = tf.convert_to_tensor(X_train, dtype=tf.float32) / 255.
8 dataset = tf.data.Dataset.from_tensor_slices((X_train, y_train))
9 dataset = dataset.batch(32).repeat(10)
10 # 获取图片的大小
11 in_shape = X_train.shape[1:]      # 形状为(28, 28)
12 # 获取数字图片的种类
13 n_classes = len(np.unique(y_train)) # 类别数为10
14
15 model = Sequential() # 搭建空顺序模型
16 model.add(layers.Flatten(input_shape=in_shape))
17 model.add(layers.Dense(n_classes, activation='softmax'))
18 # 设置优化器, 学习率设置为0.01
19 optimizer = optimizers.SGD(lr=0.01)
20 # 设置算法性能的评估标准
21 acc_meter = metrics.Accuracy()
22 for step, (x, y) in enumerate(dataset):
23
24     with tf.GradientTape() as tape:
25         # 计算模型输出
26         out = model(x)
27         # 将标签变成独热编码
28         y_onehot = tf.one_hot(y, depth=10)
29         # 计算损失
30         loss = tf.square(out - y_onehot)
31         # 计算损失均值
32         loss = tf.reduce_sum(loss) / 32
33     acc_meter.update_state(tf.argmax(out, axis=1), y)
34     grads = tape.gradient(loss, model.trainable_variables)
35     optimizer.apply_gradients(zip(grads, model.trainable_variables))
36     if step % 200 == 0:
37         print('step {0}, loss: {1:.3f}, acc: {2:.2f} %'.format(step, float(loss),
38             acc_meter.result().numpy() * 100))
39     acc_meter.reset_states()

```

图 4 构建梯度递减模型

图 4 所示代码这段代码首先导入了必要的库，包括 NumPy 和 TensorFlow，以及 Keras 中的相关模块，如数据集、层、优化器、顺序模型和性能评估标准。接着，使用 `datasets.mnist.load_data()` 方法加载 MNIST 手写数字数据集，并将其分为训练集(`X_train, y_train`)和测试集(`X_test, y_test`)，其中 `X_train` 和 `X_test` 包含图像数据，而 `y_train` 和 `y_test` 包含对应的标签。

随后，训练集的图像数据通过 `tf.convert_to_tensor` 方法转换为 TensorFlow 张量，并通过除以 255 将像素值归一化到 0 到 1 的范围，从而提高模型训练的效果。

接下来，代码创建了一个 TensorFlow 数据集对象，使用 `tf.data.Dataset.from_tensor_slices` 将训练数据和标签组合在一起，并将数据集分为批次，每批大小为 32，同时通过 `.repeat(10)` 方法使数据集在训练过程中重复 10

次，以确保模型能够多次遍历整个训练集。然后，代码获取输入图像的形状，存储在 `in_shape` 变量中，形状为(28,28)，并获取数字图像类别数，存储在 `n_classes` 变量中，该值为 10，因为 MNIST 数据集中包含的数字类别为 0 到 9。接着，使用 `Sequential` 类构建了一个顺序模型，并添加了一个 `Flatten` 层，将输入图像展平为一维数组，以便后续的全连接层处理，然后添加一个全连接层，输出神经元的数量等于类别数，并使用 `softmax` 激活函数，将输出转化为概率分布。随后，代码设置优化器为随机梯度下降（SGD），学习率设置为 0.01，并初始化一个准确率评估标准 `acc_meter`。使用 `enumerate` 遍历数据集，以批次获取图像和标签。在每个步骤中，通过 `tf.GradientTape` 记录计算过程，计算模型的输出，并将标签转换为独热编码格式，以便与模型输出进行比较。

（5）构建带有编译参数的模型，示例代码如图 5。

```
1 import numpy as np
2 from tensorflow.keras.datasets import mnist
3 import tensorflow as tf
4 from tensorflow import keras
5 def preprocess(x, y): # 自定义的预处理函数
6     # 调用此函数时会自动传入x,y 对象, shape 为[b, 28, 28], [b]
7     # 标准化到0~1
8     x = tf.cast(x, dtype=tf.float32) / 255.
9     x = tf.reshape(x, [-1, 28*28]) # 打平
10    y = tf.cast(y, dtype=tf.int32) # 转成整型张量
11    y = tf.one_hot(y, depth=10) # one-hot 编码
12    # 返回的x,y 将替换传入的x,y 参数, 从而实现数据的预处理功能
13    return x,y
14 # 导入数据
15 (X_train,y_train), (X_test, y_test) = mnist.load_data()
16
17 # 获取图片的大小
18 in_shape = X_train.shape[1:] # 形状为(28, 28)
19 # 获取数字图片的种类
20 n_classes = len(np.unique(y_train)) #类别数为10
21
22 #数据预处理, 将0~255缩放到0~1范围
23 x_train = X_train.astype('float32') / 255.0
24 x_test = X_test.astype('float32') / 255.0
25
26 model = keras.Sequential() #搭建空顺序模型
27 model.add(keras.layers.Flatten(input_shape=in_shape))
28 model.add(keras.layers.Dense(n_classes, activation='softmax'))
29
30 #编译模型: 定义损失函数和优化函数
31 model.compile(optimizer='adam',
32               loss='sparse_categorical_crossentropy',
33               metrics=['accuracy'])
```

图 5 线性回归

图 5 所示代码使用对于多分类问题 `sparse_categorical_crossentropy` 进行分类的预测；若是稀疏数据，如手写数字识别，其预测的独热编码中，10 个向量中

只有一位是 1,1 在众多的 0 中显得较为稀疏，所以使用前述损失函数较好。

(6) 进行训练和预测。示例代码如图 6。

```
1 # 模型拟合
2 model.fit(x_train, y_train, epochs=10, batch_size=128, verbose=0)
3 # 评估模型
4 loss, acc = model.evaluate(x_test, y_test, verbose=0)
5 print('测试集的预测准确率: {0:.3f}'.format(acc))
6
7 # 单个图片预测
8 image = x_train[100]      # 选择第101图做测试
9 import tensorflow as tf
10 image = tf.expand_dims(image, axis = 0)
11
12 yhat = model.predict([image])
13 print('预测的数字为: {0}'.format(np.argmax(yhat)))
14 # 绘制像素数据
15 plt.title('label = {}'.format(y_train[0]))
16 #plt.imshow(x_train[100], cmap=plt.get_cmap('gray_r')) #try: plt.get_cmap('gray'), 黑底白字, #'seismic' 彩色
17 plt.imshow(x_train[100], cmap=plt.get_cmap('gray'))
18 #plt.axis('off')
19 # 显示图片
20 plt.show()
```

图 6 训练与预测

图 6 所示代码中首先，使用 `model.fit()` 方法对模型进行训练，拟合训练数据 `x_train` 和对应的标签 `y_train`。这里设置了训练的轮数（`epochs=10`），表示整个训练集将被遍历 10 次，同时设置批次大小为 128（`batch_size=128`），表示每次更新模型参数时使用 128 个样本。`verbose=0` 表示不输出训练过程中的详细信息。

接下来，使用 `model.evaluate()` 方法对模型进行评估，计算在测试集 `x_test` 和 `y_test` 上的损失和准确率。`verbose=0` 选项同样表示不输出评估过程中的详细信息。评估结果被存储在 `loss` 和 `acc` 变量中，其中 `acc` 表示模型在测试集上的准确率。然后，使用 `print()` 函数输出测试集的预测准确率，格式化为小数点后三位。

接下来，代码选择训练集中的第 101 张图像作为测试样本，并将其存储在变量 `image` 中。为了使该图像符合模型输入的要求，使用 `tf.expand_dims(image,axis=0)` 方法在图像的最前面添加一个维度，将其形状从 (28,28) 转换为 (1,28,28)，这样可以将其视为一个批次的输入。

然后，使用 `model.predict([image])` 方法对这个单独的图像进行预测，返回的结果是一个包含每个类别概率的数组，存储在变量 `yhat` 中。通过 `np.argmax(yhat)` 获取预测结果中概率最大的类别索引，并使用 `print()` 函数输出预测的数字。

接着，绘制该图像的像素数据。使用 `plt.title()` 设置图像的标题，显示该图像

的真实标签。然后，通过 `plt.imshow()` 方法绘制图像，设置颜色映射为灰度（`cmap=plt.get_cmap('gray')`），这使得图像以黑底白字的方式显示。最后，使用 `plt.show()` 显示绘制的图像。

（7）模型的保存。示例代码如图 7。

```
1 # 将模型结构序列化为JSON格式
2 model_json = model.to_json()
3 with open("model.json", "w") as json_file:
4     json_file.write(model_json)
5 #将模型权值序列化HDF5格式
6 model.save_weights("model.h5")
7 print("成功：将模型保存本地！")
```

图 7 保存模型

图 7 所示代码中，首先，使用 `model.to_json()` 方法将模型的结构序列化为 JSON 格式的字符串。这个字符串包含了模型的层次结构、每层的参数、激活函数等信息，但不包括权重。接下来，通过 `with open("model.json","w") as json_file:` 语句打开一个名为 `model.json` 的文件，以写入模式("w")打开它。然后，使用 `json_file.write(model_json)` 将之前生成的 JSON 字符串写入该文件中。这一过程完成后，模型的结构就被保存到了 `model.json` 文件中。

接着，使用 `model.save_weights("model.h5")` 方法将模型的权重保存为 HDF5 格式的文件，命名为 `model.h5`。HDF5 是一种用于存储大量数据的文件格式，能够有效地存储模型的权重参数。

（8）通过序列化重构模型。代码如图 8 所示


```

1 #读入模型文件
2 json_file = open('model.json', 'r')
3 loaded_model_json = json_file.read()
4 json_file.close()
5 #反序列化：导入模型拓扑结构
6 loaded_model = model_from_json(loaded_model_json)
7 # 反序列化：将权值导入到加载的模型中
8 loaded_model.load_weights("model.h5")
9 print("成功：从本地文件中导入权值参数！")
10 #编译导入的模型
11 loaded_model.compile(optimizer='adam',
12                       loss='sparse_categorical_crossentropy',
13                       metrics=['accuracy'])
14 # 测试模型是否可用
15 loss, acc = loaded_model.evaluate(x_test, y_test, verbose=0)
16 print('测试集合的预测准确率: {0:.2f} %'.format(acc * 100))

```

图 8 反序列化模型

图 8 所示代码中从保存的模型文件中加载神经网络模型的结构和权重，以便进行后续的推理或评估。

首先，使用 `open('model.json','r')` 打开名为 `model.json` 的文件，以读取模式('r') 打开它，并将文件对象保存在 `json_file` 变量中。接着，使用 `json_file.read()` 读取文件内容，这将返回一个包含模型结构的 JSON 字符串，并将其存储在 `loaded_model_json` 变量中。随后，调用 `json_file.close()` 关闭文件，释放资源。

接下来，使用 `model_from_json(loaded_model_json)` 函数反序列化模型结构，创建一个新的模型实例 `loaded_model`，这个模型的架构与之前保存的模型相同，但并没有权重。

然后，使用 `loaded_model.load_weights("model.h5")` 方法将之前保存的权重加载到新创建的模型中，这样 `loaded_model` 便具备了原模型的参数和训练状态。成功加载权重后，代码输出一条消息"成功：从本地文件中导入权值参数！"，以确认操作成功。

接下来，代码调用 `loaded_model.compile()` 方法对加载的模型进行编译，设置优化器为 `adam`，损失函数为 `sparse_categorical_crossentropy`（适用于多类分类问题，且标签为整数格式），并指定评估指标为准确率(`metrics=['accuracy']`)。

最后，代码使用 `loaded_model.evaluate(x_test,y_test,verbose=0)` 方法在测试集上评估加载后的模型并进行评估。